PKU–Zurich PhD Summer School on Machine Learning for Macroeconomics and Finance

From representative-agent Brock–Mirman through stochastic models, KKT constraints, and loss kernels

Simon Scheidegger

HEC Lausanne · University of Zurich

July 6–10, 2026 · Beijing, China

Based on: Azinovic, Gaegauf & Scheidegger (2022), *International Economic Review* 63(4), 1471–1525.

https://liboecon.com/2026_summer_school.html

Roadmap of This Lecture

Motivation

- Heterogeneity and high dimensions
- Nonlinearities and kinks
- Irregular ergodic sets
- Our solution: DEQNs

From ANNs to DEQNs

- DNNs as function approximators
- Supervised vs. self-supervised loss
- The DEQN training algorithm

Brock–Mirman Benchmark

- Analytical test case
- DEQN implementation

Known Pathologies (& Remedies)

- Loss imbalance, architecture, lazy learning, residual learning, coverage

Parallelization & HPC

- Data parallelism (MPI/Horovod); scalability to many nodes

Wrap-up + Hands-on

- Notebooks 01_01–01_04 (BM, BM+shocks, exercises); optional 01_05
- Exercise preview: KKT + Fischer–Burmeister, 6-period OLG

Learning objectives

By the end of this lecture you should be able to:

- Formulate dynamic stochastic economic models as equilibrium systems and identify computational challenges in high dimensions
- Replace grid-based collocation with self-supervised neural-network training using equilibrium errors as the loss function
- Build and train Deep Equilibrium Nets (DEQNs) to solve representative-agent and heterogeneous-agent models
- Implement simulated paths from the model's ergodic distribution to construct training data without explicit grids
- Parallelize DEQN training across multiple nodes and scale solutions to 100+ dimensions

Part I

Motivation

Heterogeneity Matters

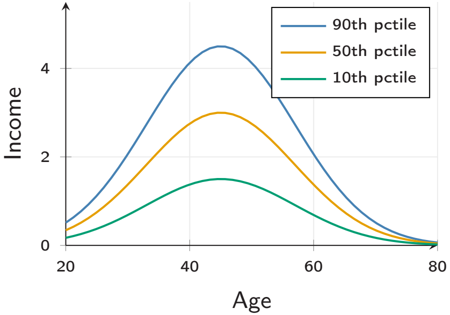
Key questions: inequality over the life cycle; idiosyncratic vs. aggregate risk; distributional effects of policy.

Quantitative punchline:

- Hand-to-mouth HH: MPC ≈ 1; unconstrained: MPC ≈ 0.05
- Representative agent averages these out, misses the policy response

Kaplan, Moll & Violante 2018; Bilbiie 2008; Krueger et al. 2016

$\Rightarrow$ Heterogeneity is one of several forces that blow up d (also: many countries, many assets, climate state, ...).



Stylized income-by-age profiles

Dynamic Stochastic General Equilibrium Models

Generic formulation: find policy functions $p(\cdot)$ such that

$$\mathbb{E}\Big[G\big(\boldsymbol{x}_t,\, \boldsymbol{x}_{t+1},\, p(\boldsymbol{x}_t),\, p(\boldsymbol{x}_{t+1})\big)\;\Big|\; \boldsymbol{x}_t,\, p(\boldsymbol{x}_t)\Big] = 0$$

with transition law $\boldsymbol{x}_{t+1} = h\big(\boldsymbol{x}_t,\, p(\boldsymbol{x}_t),\, \varepsilon_{t+1}\big)$.

Four computational challenges (Azinovic, Gaegauf & Scheidegger, 2022):

1. Stochasticity: nontrivial expectations over ε_{t+1} and the ergodic distribution
2. High-dimensional state space: heterogeneity, many assets, many shocks all push d up
3. Strong nonlinearities / kinks from occasionally binding constraints and regime switches
4. Irregular geometry of the ergodic set: recurrent states are not a hypercube

No classical method addresses all four jointly: sparse grids cover (1–3) but become impractical past $d \sim 20$ under conventional smoothness assumptions; Gaussian processes cover (1,2,4) but their $O(N^3)$ training cost limits them to small N.

What is High-Dimensional?

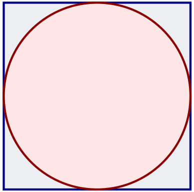
Dimensions d	Grid points n^d	Runtime
1	10	10 seconds
2	100	2 minutes
5	10^5	1 day
10	10^{10}	317 years
20	10^{20}	$\sim$ 3 trillion years

Assumes $n = 10$ grid points per dimension and 1 second per evaluation.

Three remedies:

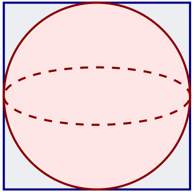
1. Sparse grids (Brumm & Scheidegger, 2017)
2. Random grids (Rust, 1997)
3. Neural networks (this lecture)

Volumes in High Dimensions



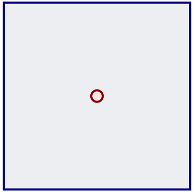
2D

$V_{\text{sphere}}/V_{\text{cube}} = \pi/4 \approx 0.785$



3D

$V_{\text{sphere}}/V_{\text{cube}} = \pi/6 \approx 0.524$



100D

$V_{\text{sphere}}/V_{\text{cube}} \approx 1.87 \times 10^{-70}$

Implication

In high dimensions, almost all volume is in the corners. Grid-based methods waste most of their evaluation points in regions that barely contribute to the integral or approximation.

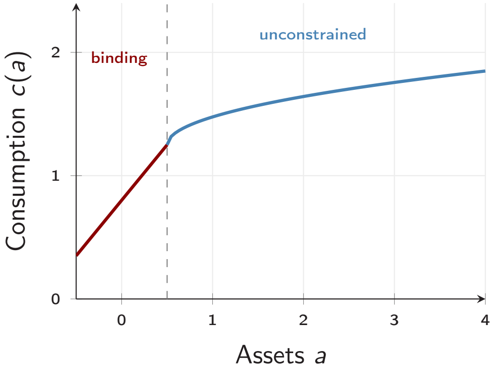
Nonlinearities and Kinks

Where do kinks come from?

- Borrowing constraints: $a_{t+1} \geq \underline{a}$
- Zero lower bound: $i_t \geq 0$
- Short-sale / participation: corner solutions
- Default, regime switches, indivisibilities

Occasionally binding constraints $\Rightarrow$ policy functions are non-smooth at the boundary of the binding region.

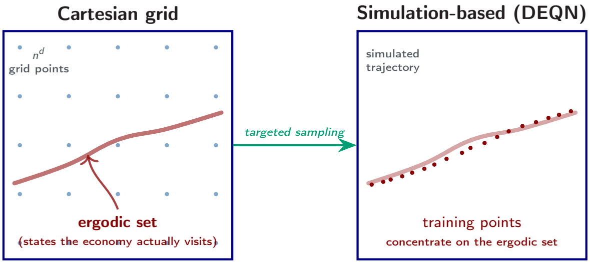
Polynomial / spectral methods lose accuracy; sparse grids can resolve kinks but only for $d \lesssim 20$. Neural networks handle kinks natively via ReLU-type activations.



Kinked consumption policy at the borrowing limit

The Ergodic Set Is Not a Hypercube

Grid-based methods tile the full hypercube $\boldsymbol{x} \in [\boldsymbol{x}_{\min}, \boldsymbol{x}_{\max}]^d$. **But** simulated states drawn from the model's law of motion fill only a thin, irregular subset, the ergodic set (states actually visited by the economy), often $\ll 1\%$ of the hypercube (Judd, Maliar & Maliar, 2011).



A full Cartesian grid wastes most of its work in regions the economy never visits $\Rightarrow$ motivates simulation-based, grid-free methods.

Abstract Problem Formulation

We need a numerical method that satisfies:

1. **Global approximation:** avoid purely local perturbation solutions
2. **Approximate solutions:** accept small residual error $\varepsilon > 0$
3. **Grid-free in practice:** avoid explicit tensor-product growth n^d
4. **Flexible geometry:** handle nonlinearities, kinks, and irregular ergodic sets
5. **Parallel-friendly:** benefit from parallelization on modern hardware

$\Rightarrow$ **DEQNs address this wishlist well in practice,** while mitigating rather than eliminating high-dimensional costs.

Our Solution: Deep Equilibrium Nets

Core idea (Azinovic, Gaegauf & Scheidegger, 2022):

Replace the traditional grid-based solution with a neural network trained to satisfy economic equilibrium conditions.

Three Key Ingredients

1. **Equilibrium error as loss function:** use the economic equations $G(\cdot) = 0$ directly, with no labeled data needed
2. **Stochastic gradient descent:** optimize network parameters θ via SGD on mini-batches
3. **Simulated paths:** draw training points from the ergodic distribution of the model itself

$\Rightarrow$ This turns solving an economic model into training a neural network.

It avoids explicit state-space grids, but expectation accuracy, sampling quality, and optimization still matter.

Part II

From ANNs to Deep Equilibrium Nets

What is a Deep Neural Network?

A deep neural network (DNN) $\mathcal{N}_\theta : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$ is a parametric function approximator.

Universal approximation (Cybenko 1989, Hornik et al. 1989):

$$\forall f \in C(\mathcal{K}, \mathbb{R}),\ \forall \varepsilon > 0,\ \ \exists\, \theta^\star : \quad \sup_{\boldsymbol{x} \in \mathcal{K}} \left\| \mathcal{N}_{\theta^\star}(\boldsymbol{x}) - f(\boldsymbol{x}) \right\| < \varepsilon$$

($\mathcal{K} \subset \mathbb{R}^{d_{\mathrm{in}}}$ compact; the norm is the supremum over $\mathcal{K}$.)

In economics:

- Input: state variables $\boldsymbol{x}_t \in \mathbb{R}^d$ (capital, productivity, wealth distribution, ...)
- Output: policy variables (consumption, savings, prices, ...)
- Parameters θ: weights and biases, found by training

DNN Layer Structure

$$\underbrace{\boxed{\boldsymbol{x} \in \mathbb{R}^{d_{\text{in}}}}}_{\text{Input}} \longrightarrow \underbrace{\boxed{\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}}}_{\text{Hidden 1}} \longrightarrow \underbrace{\boxed{\boldsymbol{W}^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)}}}_{\text{Hidden 2}} \longrightarrow \cdots \longrightarrow \underbrace{\boxed{\boldsymbol{W}^{(L)}\boldsymbol{a}^{(L-1)} + \boldsymbol{b}^{(L)}}}_{\text{Output}}$$

Each layer performs an affine transformation:

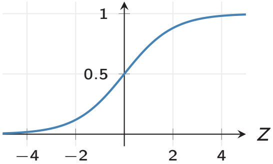
$$\boldsymbol{z}^{(\ell)} = \boldsymbol{W}^{(\ell)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}, \qquad \ell = 1, \ldots, L$$

where $\boldsymbol{W}^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ and $\boldsymbol{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$.

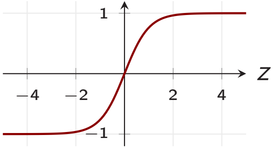
Without nonlinear activations, the entire network collapses to a single affine map.

Activation Functions

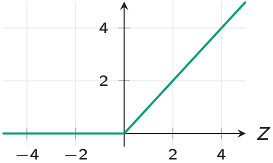
Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$



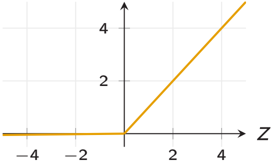
Tanh: $\tanh(z)$



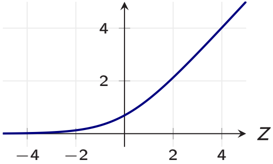
ReLU: $\max(0, z)$



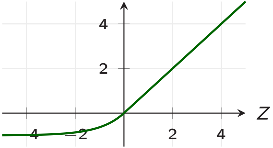
Leaky ReLU: $\max(0.01z, z)$



Softplus: $\ln(1 + e^{z})$



ELU: z if $z \geq 0$; $e^{z} - 1$ if $z < 0$



DNN with Nonlinear Activations

Input	Hidden 1	Hidden 2	Output (no activation)
$\boldsymbol{x} \in \mathbb{R}^{d_{\text{in}}}$ $\rightarrow$	$\phi^{(1)}\big(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\big)$ $\rightarrow$	$\phi^{(2)}\big(\boldsymbol{W}^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)}\big)$ $\rightarrow$	$\boldsymbol{W}^{(L)}\boldsymbol{a}^{(L-1)} + \boldsymbol{b}^{(L)}$

Each hidden layer applies a nonlinear activation $\phi^{(\ell)}$:

$$\boldsymbol{a}^{(\ell)} = \phi^{(\ell)}\big(\boldsymbol{W}^{(\ell)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}\big), \qquad \ell = 1, \dots, L-1$$

- The output layer is typically linear (for regression) or uses softplus (to enforce positivity, e.g. consumption)
- The full network: $\mathcal{N}_\theta(\boldsymbol{x}) = \boldsymbol{W}^{(L)}\boldsymbol{a}^{(L-1)} + \boldsymbol{b}^{(L)}$

Finding Parameters: Supervised Learning

Standard approach (when labeled data $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$ are available):

1. **Collect labeled data:** $\boldsymbol{x}_i \xrightarrow{\text{true mapping}} y_i$
2. **Define a loss function:**

$$\ell_\theta^{\text{sup}} = \frac{1}{N}\sum_{i=1}^{N} \left\| y_i - \mathcal{N}_\theta(\boldsymbol{x}_i) \right\|^2 \qquad \text{(mean squared error)}$$

3. **Update via SGD** ($\eta =$ learning rate; the symbol α is reserved for the Cobb–Douglas capital share later in the deck):

$$\theta^{\text{new}} = \theta^{\text{old}} - \eta \cdot \left.\frac{\partial \ell_\theta}{\partial \theta}\right|_{\theta=\theta^{\text{old}}}$$

This is the recipe from Day 1's machine-learning introduction. But in economics, we rarely have labeled data...

The "Data Problem" in Economics

Deep learning excels when massive labeled datasets are available.

But in computational economics:

- We solve functional equations, not classification/regression tasks
- There are no labeled pairs $(\boldsymbol{x}_i, y_i)$
- Traditional approach: time iteration on a grid, but grids do not scale

Key insight:

- We know the structural equations $G(\cdot) = 0$
- We can check how well a candidate policy satisfies them
- $\Rightarrow$ Use the equilibrium error as the loss function!

Traditional Approach: Time Iteration on a Grid

Algorithm: Grid-Based Collocation

Input: Grid $\mathcal{G} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_M\} \subset \mathbb{R}^d$, initial guess $p^{(0)}$
for $k = 0, 1, 2, \ldots$ until convergence **do**
  for each grid point $\boldsymbol{x}_i \in \mathcal{G}$ **do**
    Solve $G\big(\boldsymbol{x}_i,\ p^{(k)}(\boldsymbol{x}_i),\ p^{(k)}\big) = 0$ for $p^{(k+1)}(\boldsymbol{x}_i)$
  end for
  Interpolate $p^{(k+1)}$ between grid points
end for

Problems:

- Cartesian grids need $M = n^d$ points, infeasible beyond $d \sim$ 4–5 at sensible resolution (10^5 at $d = 5$, 10^{10} at $d = 10$)
- Sparse grids (Smolyak, adaptive) extend the practical reach only to $d \sim$ 20–50
- Heterogeneous-agent or many-country DSGE models routinely need $d \gg 50$, beyond any grid-based method

The DEQN Loss Function

Key idea: replace labeled data with economic equilibrium conditions.

Let $G(\boldsymbol{x}, f(\boldsymbol{x})) = 0$ be the system of equilibrium equations. Define:

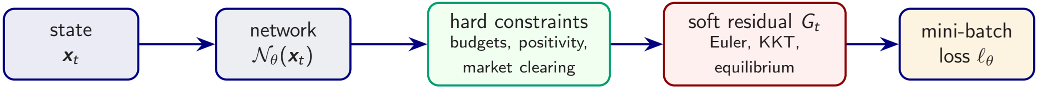
$$\boxed{\ell_\theta = \frac{1}{N}\sum_{i=1}^{N}\left\| G\left(\boldsymbol{x}_i, \mathcal{N}_\theta(\boldsymbol{x}_i)\right)\right\|^2}$$

- $\mathcal{N}_\theta(\boldsymbol{x}_i)$ is the neural net's predicted policy at state $\boldsymbol{x}_i$
- $G(\cdot)$ measures the equilibrium error (e.g. Euler equation residual)
- Small ℓ_θ means the policy nearly satisfies the equations on the sampled states

$\Rightarrow$ No labels needed; we call this **self-supervised** learning (sometimes labelled "unsupervised" in the narrow no-labels sense).

In stochastic models, expectations inside G are approximated, e.g. by path averaging or quadrature.

Inside One DEQN Training Step



- Encode what can be enforced exactly in the architecture.
- Minimize only the genuinely non-closed-form equilibrium conditions.
- Draw states from the model's own simulation, so training focuses on the ergodic set.

Training Algorithm for DEQNs

Algorithm 1: Deep Equilibrium Net Training

Input: Initial state $\mathbf{x}_0$, network $\mathcal{N}_\theta$, learning rate η, episodes E, simulation horizon T_{sim}, training steps T_{train}

for episode $e = 1, \ldots, E$ **do**

  Simulate path: $\mathbf{x}_0 \to \mathbf{x}_1 \to \cdots \to \mathbf{x}_{T_{\mathrm{sim}}}$ using $\mathcal{N}_\theta$ and transition law[1]

  for gradient step $t = 1, \ldots, T_{\mathrm{train}}$ **do**

    Draw mini-batch $\mathcal{B} \subset \{\mathbf{x}_0, \ldots, \mathbf{x}_{T_{\mathrm{sim}}}\}$

    Compute loss: $\ell_\theta = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x}_i \in \mathcal{B}} \|G(\mathbf{x}_i, \mathcal{N}_\theta(\mathbf{x}_i))\|^2$

    Update: $\theta \leftarrow \theta - \eta \cdot \nabla_\theta \ell_\theta$

  end for

end for

Output: Trained network $\mathcal{N}_{\theta^\star}$ approximating the policy function

Note: Training points come from the model's own simulation, so the network learns on the ergodic distribution, focusing on economically relevant regions.

[1] The IRBC/OLG notebooks (Sessions 3–4) use a *persistent* variant: trajectories continue across episodes, with no reset.

Supervised vs. Self-Supervised (DEQN) Learning

Supervised

Labeled data $(\boldsymbol{x}_i, y_i)$

$\ell = \|y_i - \mathcal{N}_\theta(\boldsymbol{x}_i)\|^2$

SGD $\to \theta^\star$

DEQN (Self-Supervised)

Equations $G(\boldsymbol{x}, p(\boldsymbol{x})) = 0$

$\ell = \|G(\boldsymbol{x}_i, \mathcal{N}_\theta(\boldsymbol{x}_i))\|^2$

SGD $\to \theta^\star$

Key difference: DEQNs replace labeled data with structural economic equations.

DEQN: Key Recap

Replace Labels

Instead of labeled data $(\boldsymbol{x}_i, y_i)$, use the economic equilibrium equations $G(\cdot) = 0$ as the loss.

Replace Grids

Instead of n^d grid points, use a neural network $\mathcal{N}_\theta$ that maps states to policies globally.

Replace TI

Instead of time iteration with interpolation, use SGD on simulated paths from the model.

Result: A grid-free, simulation-based method that can handle much higher-dimensional models than standard grids and maps naturally to GPUs.

Part III

Brock–Mirman Benchmark

Brock–Mirman (1972): Setup

Social planner's problem. Choose a consumption sequence to maximize expected lifetime log utility:

$$\max_{\{C_t\}_{t=0}^{\infty}} \mathbb{E}\left[\sum_{t=0}^{\infty} \beta^t \ln C_t\right]$$

Resource constraint (full depreciation, $\delta = 1$):

$$C_t + K_{t+1} = z_t K_t^{\alpha}, \qquad K_0 > 0 \text{ given.}$$

Stochastic productivity. TFP z_t follows an AR(1) in logs:

$$\ln z_{t+1} = \varrho \ln z_t + \sigma_z \epsilon_{t+1}, \qquad \epsilon_{t+1} \sim \mathcal{N}(0, 1).$$

State & control. State $\boldsymbol{x}_t = (K_t, z_t)$; control C_t. Capital then follows from the budget as $K_{t+1} = z_t K_t^{\alpha} - C_t$.

Next two slides: derive the Euler equation from first principles, then obtain the closed-form policy – the benchmark we will later compare the DEQN solution to.

Deriving the Euler Equation (Lagrangian)

Step 1. Lagrangian. Attach a multiplier $\lambda_t \geq 0$ to each period's budget constraint:

$$\mathcal{L} \;=\; \mathbb{E}\left[\sum_{t=0}^{\infty} \beta^t \Big[\, \ln C_t \;-\; \lambda_t \left(C_t + K_{t+1} - z_t K_t^{\alpha}\right)\Big]\right].$$

Step 2. Take partial derivatives with respect to the choices at date t:

Consumption FOC $\partial\mathcal{L}/\partial C_t = 0$:

$$\beta^t \left(\frac{1}{C_t} - \lambda_t\right) = 0 \;\;\Longrightarrow\;\; \lambda_t = \frac{1}{C_t}.$$

Capital FOC $\partial\mathcal{L}/\partial K_{t+1} = 0$:

$$-\beta^t \lambda_t \;+\; \beta^{t+1}\,\mathbb{E}\big[\lambda_{t+1}\,\alpha z_{t+1} K_{t+1}^{\alpha-1}\big] = 0.$$

Step 3. Eliminate λ. Substitute $\lambda_t = 1/C_t$, $\lambda_{t+1} = 1/C_{t+1}$ and divide by β^t:

$$\boxed{\frac{1}{C_t} \;=\; \beta\,\mathbb{E}\left[\frac{\alpha\, z_{t+1}\, K_{t+1}^{\alpha-1}}{C_{t+1}}\right]} \qquad \text{(stochastic Euler equation)}$$

Reading: marginal utility of today's consumption $=$ discounted expected marginal utility of tomorrow's consumption $\times$ gross marginal product of capital at $t{+}1$. The transversality condition $\lim_{t\to\infty} \mathbb{E}[\beta^t \lambda_t K_{t+1}] = 0$ rules out bubbles.

Closed-Form Policy via Guess-and-Verify

Guess. Consumption is a constant fraction $1-\phi$ of output:

$$C_t \;=\; (1-\phi)\, z_t K_t^{\alpha}, \qquad \phi \in (0,1) \text{ to be determined.}$$

The budget then implies $K_{t+1} = \phi\, z_t K_t^{\alpha}$.
Substitute into the Euler equation. With $C_{t+1} = (1-\phi)\, z_{t+1} K_{t+1}^{\alpha}$,

$$\frac{\alpha\, z_{t+1}\, K_{t+1}^{\alpha-1}}{C_{t+1}} = \frac{\alpha\, z_{t+1}\, K_{t+1}^{\alpha-1}}{(1-\phi)\, z_{t+1}\, K_{t+1}^{\alpha}} = \frac{\alpha}{(1-\phi)\, K_{t+1}}.$$

Note: z_{t+1} cancels, so the conditional expectation collapses (log + Cobb–Douglas + full depreciation).
Solve for ϕ. Plug $K_{t+1} = \phi z_t K_t^{\alpha}$ on both sides of the Euler:

$$\underbrace{\frac{1}{(1-\phi)\, z_t K_t^{\alpha}}}_{\text{LHS}} = \beta \cdot \underbrace{\frac{\alpha}{(1-\phi)\, \phi\, z_t K_t^{\alpha}}}_{\text{RHS}} \implies 1 = \frac{\beta\alpha}{\phi} \implies \phi = \beta\alpha.$$

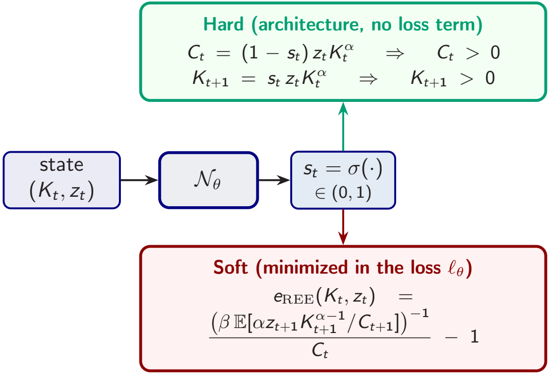
Closed-form policy.

$$\boxed{C_t \;=\; (1-\beta\alpha)\, z_t K_t^{\alpha}, \qquad K_{t+1} \;=\; \beta\alpha\, z_t K_t^{\alpha}}$$

This benchmark provides an analytic target against which we will measure the DEQN's trained policy.

Brock–Mirman: Hard vs. Soft Constraints

Split the equilibrium conditions by what the architecture can enforce exactly.



Why bother? A softplus head on C_t alone would secure $C_t > 0$ but could push $K_{t+1} < 0$. The sigmoid-savings head removes *both* failure modes before training starts. Only the genuinely non-closed-form condition, the Euler equation, enters the loss.

Brock–Mirman: Residual, Expectation, Loss

Pathwise residual used in the notebook (full-depreciation $\delta = 1$ benchmark; G_t is the pathwise counterpart of the e_{REE} shown two slides back — both share the same zero set):

$$G_t = 1 - \beta \frac{C_t}{C_{t+1}} \cdot \alpha z_{t+1} K_{t+1}^{\alpha-1},$$

with

$$K_{t+1} = s_t z_t K_t^{\alpha}, \qquad C_{t+1} = (1 - s_{t+1}) z_{t+1} K_{t+1}^{\alpha}.$$

Partial depreciation. The uncertainty notebook uses $\delta = 0.1$; the marginal productivity of capital then becomes $\alpha z_{t+1} K_{t+1}^{\alpha-1} + (1-\delta)$ and $K_{t+2} = z_{t+1} K_{t+1}^{\alpha} + (1-\delta) K_{t+1} - C_{t+1}$. The closed-form $\phi = \alpha\beta$ only holds at $\delta = 1$.

Training loss:

$$\ell_\theta = \frac{1}{N} \sum_{i=1}^{N} |G_t^{(i)}|^2.$$

- Simulate states (K_t, z_t) forward
- At each state, take $\mathbb{E}_t[\cdot]$ over z_{t+1} by 5-node Gauss–Hermite quadrature
- Average G_t^2 over many simulated states

Expectation Handling

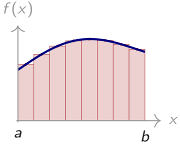
Brock–Mirman notebook: Gauss–Hermite quadrature (5 nodes) for the conditional expectation at each state.

Richer models: same idea with more nodes, or product / sparse grids in higher dimensions, see next three slides.

Numerical Integration: Why and How

Why an integral always shows up. Euler equations, Bellman operators, REE diagnostics, climate damages, structural moments, all reduce to evaluating $\mathbb{E}[g(\varepsilon)] = \int g(\varepsilon)\,\mu(\varepsilon)\,d\varepsilon$. Closed forms are rare, so we approximate by $\sum_q w_q\, g(\varepsilon_q)$. Two paradigms underlie every rule.

Deterministic: tile the area, integral $\to$ sum

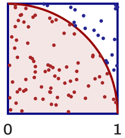


With $\Delta x = (b-a)/N$, $x_i = a + i\Delta x$, $I = \int_a^b f(x)\,dx \approx \sum_i w_i\, f(x_i)$:

- Midpoint: $I_N^{\textbf{mid}} = \Delta x \sum_{i=1}^{N} f(x_i^{\textbf{mid}})$, $\mathcal{O}(N^{-2})$
- Trapezoid: $I_N^{\textbf{trap}} = \frac{\Delta x}{2}\left[f(x_0) + 2\sum_{i=1}^{N-1} f(x_i) + f(x_N)\right]$, $\mathcal{O}(N^{-2})$
- Simpson: parabolic arcs through triples, $\mathcal{O}(N^{-4})$
- Gauss–Hermite: optimal nodes/weights for e^{-x^2}, $\mathcal{O}(N^{-(2Q-1)})$

d-dim Cartesian grid: cost $\mathcal{O}(N^d)$, curse of dimensionality.

Random: throw darts to estimate π



Throw N uniform darts at $\Omega = [0,1]^2$:

$$\frac{\pi}{4} = \int_0^1\!\!\int_0^1 1[x^2 + y^2 \le 1]\,dx\,dy \approx \frac{N_{\rm in}}{N}$$

$$\widehat{\pi}_N = 4N_{\rm in}/N, \quad \text{error } \mathcal{O}(N^{-1/2})$$

Independent of dimension, but slow.

Quadrature, I: Gauss–Hermite

Setup. Generic shock dimension d, with $\varepsilon' \sim \mathcal{N}(0, I_d)$. Brock–Mirman has $d = 1$ (one productivity shock); the IRBC session (Session 4) will use $d = N + 1$ (N country shocks plus one aggregate). We develop the framework here so it is ready for both cases.

One dimension. Gauss–Hermite uses Q nodes $\{\varepsilon_q\}$ (Hermite-polynomial roots) and weights $\{w_q\}$:

$$\mathbb{E}[h(\varepsilon)] = \int_{-\infty}^{\infty} h(\varepsilon) \frac{e^{-\varepsilon^2/2}}{\sqrt{2\pi}} d\varepsilon \approx \sum_{q=1}^{Q} w_q \, h(\varepsilon_q),$$

exact for univariate polynomials of degree $\leq 2Q - 1$.

Tensor product in d dimensions. Combine 1D rules along each axis:

$$\mathbb{E}\big[h(\varepsilon')\big] \approx \sum_{q_1=1}^{Q} \cdots \sum_{q_d=1}^{Q} \Big(\prod_j w_{q_j}\Big) \, h(\varepsilon_{q_1}, \ldots, \varepsilon_{q_d}).$$

- Strength: polynomial-exact in each coordinate, very accurate at low d. For $d = 1$ (Brock–Mirman) a $Q = 3$ rule is essentially exact.
- Weakness: cost Q^d blows up exponentially. For $Q = 3$, $d = 11$ (IRBC, $N = 10$): $\sim 1.8 \times 10^5$ nodes per state.

Quadrature, II: Monomial (Stroud-3) Rule

Idea. Drop full polynomial-exactness in every coordinate, keep exactness for total degree ≤ 3. Place nodes only on the principal axes of the d-dimensional shock space:

$$\boxed{\mathbb{E}\big[h(\varepsilon')\big] \;\approx\; \frac{1}{2d}\sum_{k=1}^{d}\Big[h(+\sqrt{d}\,\boldsymbol{e}_k) + h(-\sqrt{d}\,\boldsymbol{e}_k)\Big]}$$

where $\boldsymbol{e}_k$ is the k-th standard basis vector, giving $2d$ nodes at common radius $\sqrt{d}$. For Brock–Mirman ($d = 1$) this reduces to two nodes at ± 1 (exact for univariate polynomials of degree ≤ 3); for IRBC ($d = N + 1$) it gives $2(N + 1)$ nodes.

What it integrates exactly

- $\mathbb{E}[\varepsilon'_i] = \mathbb{E}[\varepsilon_i'^{\,3}] = 0$ ($\pm$-symmetry).
- $\mathbb{E}[\varepsilon'_i \varepsilon'_j] = 0$, $i \neq j$ (each node lies on one axis).
- $\mathbb{E}[\varepsilon_i'^{\,2}] = 1$ (variance preserved).

Where it stops

- $\mathbb{E}[\varepsilon_i'^{\,4}]$ gives d vs. truth 3, a controlled fourth-order bias.

Why DEQNs love it

The integrand $h(\varepsilon')$ is the policy network composed with model primitives, smooth and only mildly nonlinear. Degree-3 exactness is enough for Euler-error tolerances of $\sim 10^{-3}$ on IRBC-class problems.

(Pichler 2011; Maliar & Maliar 2014)

Quadrature, III: Cost and Rules of Thumb

Per-state quadrature cost ($Q = 3$ for the tensor-product rule):

N	shocks $N+1$	tensor product 3^{N+1}	monomial $2(N+1)$	speed-up
2	3	27	6	$\sim 5\times$
5	6	729	12	$\sim 60\times$
10	11	177,147	22	$\sim 8{,}000\times$
20	21	$\sim 10^{10}$	42	$\sim 2\times 10^{8}\times$

Default rules of thumb

- $N+1 \leq 4$: tensor-product GH, $Q=5$.
- $5 \leq N+1 \lesssim 30$: Stroud-3 monomial.
- $N+1 \gtrsim 30$ or fat-tailed integrands: QMC (Sobol + inverse-CDF).

Practical notes

- Only the quadrature kernel changes; same DEQN loss, same training loop.
- Validate by tightening the rule and checking the Euler-error histogram is stable.
- Full derivations: lecture script §3.5.

Diagnostics: How Do We Know Training Worked?

Never trust a loss curve alone. On an independent *test* simulation of the trained policy, run these four complementary checks:

1. Euler-error distribution. Histogram $\log_{10}|e_{\mathrm{REE}}|$ on train and test states; both should peak near 10^{-3} or below, with no fatter right tail on test.
2. Policy-function benchmark. Overlay the DEQN policy on the analytic $s^\star = \beta\alpha$ for Brock–Mirman; for richer models, on a trusted reference (time iteration, perturbation).
3. Ergodic-set coverage. Scatter test states in the $(K_t, \log z_t)$ plane; the cloud should trace the training ergodic set — not collapse to a point, not spread like a grid.
4. Sanity checks. Verify accounting identities (budget, market clearing, non-negativity); check that ergodic moments (mean K, $\mathrm{std}(z)$, autocorrelations) match analytic or pre-computed values.

Green flags: low Euler error, matches benchmark, ergodic coverage, identities hold. Red flags: NaNs, fat tails, clamped policy, violated identities.

When no closed-form benchmark exists, draw inspiration from the VVUQ literature — Verification, Validation & Uncertainty Quantification (Roache 1998; Oberkampf & Roy 2010; ASME V&V 10/20) — which formalises residual convergence, solution verification, and error-bound reporting for computational models.

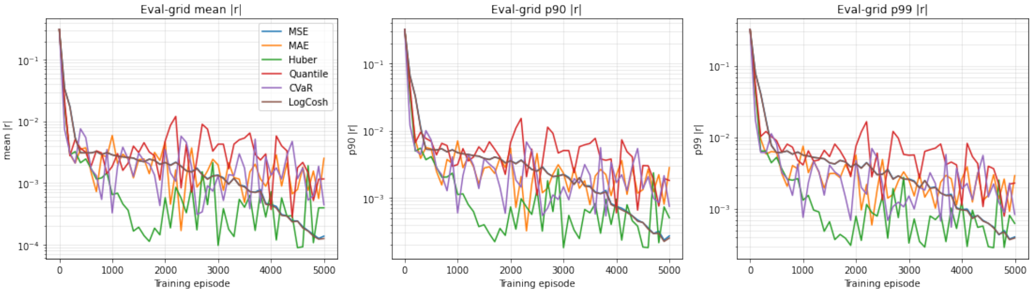
Choice of Training Loss: Six Kernels

The loss kernel is a modeling decision, not a default. Same Euler residual r, same network, same data, only the reduction $r \mapsto \ell$ changes:

Kernel	Formula	Character
MSE	$\frac{1}{B}\sum r^2$	quadratic; canonical default
MAE	$\frac{1}{B}\sum \lvert r\rvert$	constant gradient $\Rightarrow$ stalls at small $\lvert r\rvert$
Huber	quad. for $\lvert r\rvert \le \delta$, linear above	robust hybrid, but kink at $\lvert r\rvert = \delta$
Quantile	pinball at $\tau = 0.9$	trains the upper τ-quantile only
CVaR	mean of top 10% of $\lvert r\rvert$	explicitly trains the worst-case states
LogCosh	$\frac{1}{B}\sum \log\cosh(r)$	C^∞: $\frac{1}{2}r^2$ near 0, $\lvert r\rvert - \log 2$ in tails, no kink

Same Brock–Mirman model, full depreciation so $s^\star = \alpha\beta$ is in closed form; same network (2×32 swish, sigmoid head); same Adam optimizer; same CRN training stream. Six runs differ *only* in the reduction applied to the residual vector.

Loss Kernels: Convergence on Brock–Mirman



Mean / p_{90} / p_{99} of the absolute relative Euler error $|r|$ on a fixed evaluation grid, vs. training episode (log scale, 5000 episodes per kernel). LogCosh and MSE reach the lowest mean residual. Huber and CVaR produce the narrowest tail. MAE stalls at a noise floor.

Loss-Kernel Comparison: What to Take Away

- LogCosh converges most smoothly: the only C^∞ kernel that is MSE-like near $r = 0$ *and* MAE-like in the tails, no kink anywhere. Lowest-risk default when the residual distribution is unknown.
- MAE stalls at a noise floor: constant gradient magnitude prevents smaller steps as residuals shrink.
- Huber is robust but its kink at $|r|=\delta$ can introduce small oscillations.
- Quantile and CVaR narrow p_{99} at the cost of mean residual. Use them when worst-case policy quality matters (binding constraints, rare-but-consequential states, regulatory back-tests).
- Evaluate on the economic metric — Euler error along simulated paths, in CE-welfare-loss units — not on the training loss itself. The CE ranking is generally not the training-loss ranking.

Practical default

MSE remains the canonical default, but **LogCosh** is a smooth, kink-free alternative that should be benchmarked alongside it before defaulting; **Huber** / **Quantile** / **CVaR** when the worst-case state matters.

Companion notebook: `code/01_brock_mirman/01_05_StochasticBM_LossComparison.ipynb`.

Part IV

Known Pathologies of Unsupervised Deep Learning

diagnostics and remedies

Huge Potential, but Not Yet Turnkey

- **The promise:** mesh-free, high-dimensional, kinks and all.
- **The catch:** not yet a turnkey solver, and seed-sensitive.

Five pathologies of self-supervised (residual-only) training — we discuss them now, one by one:

(a) **Loss-scale imbalance** $\rightarrow$ normalize, adapt the weights.
(b) **Architecture choice** $\rightarrow$ search the candidate space.
(c) **Lazy surrogate learning** $\rightarrow$ anchors and distillation.
(d) **Learning from scratch** $\rightarrow$ freeze a guess, learn the correction.
(e) **Self-confirming coverage** $\rightarrow$ audit on a coverage measure (Equilibrium World Models).

Seed dependence

Same model, same code, different seed: one run converges and verifies, another stalls. Reproducibility is part of the method.

(a) The Residual Loss Is Not One Loss

Symptom. The training loss sums residuals that live on very different scales:

$$\ell(\boldsymbol{\theta}) = \sum_{k=1}^{K} w_k \, \ell_k(\boldsymbol{\theta}).$$

- Euler equations, KKT / complementarity, market clearing, boundary terms.
- With uniform weights the optimizer follows the **loudest** component and leaves quiet but economically important residuals large.

One loss, many voices

A small *total* loss can hide a component that is orders of magnitude too large. The total alone is not a certificate.

(a) Rebalancing the Loss

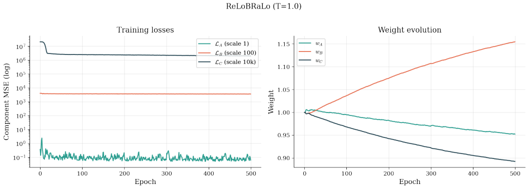
- **Fix.** Normalize each residual and **adapt the weights** during training.
- For instance, ReLoBRaLo raises the weight on terms whose relative progress lags:

$$w_k^{(t)} = \alpha\big[\rho\, w_k^{(t-1)} + (1-\rho)\,\hat{w}_{k,\mathrm{base}}^{(t)}\big] + (1-\alpha)\,\hat{w}_{k,\mathrm{step}}^{(t)}.$$

$\alpha, \rho \in [0,1]$: a random look-back weight and an EMA decay; $\hat{w}_{\mathrm{base}}$ uses the running history, $\hat{w}_{\mathrm{step}}$ the current step.

Remedy

Monitor every component; rebalance with ReLoBRaLo, SoftAdapt, or GradNorm. Session 2 develops this in detail.



One training run: separate residual components and the adaptive weights that keep them balanced.

(b) Architecture Is Part of the Numerical Method

- **Symptom.** With the *same* economic equations, one network converges and another stalls.
- Width, depth, activation, and learning rate are **numerical-method choices**, not cosmetics.

The search space. Make the candidate set explicit:

- depth $\in \{2, \ldots, 5\}$, width $\in \{32, 64, 128, 256\}$;
- activation $\in \{\mathrm{ReLU}, \mathrm{Swish}, \tanh\}$, smooth ones for second-order PDEs;
- learning rate, batch size, weight decay.
- these axes *multiply*: a **curse of dimensionality in design space**.

Hand-tuning does not scale

One architecture is one experiment. Reporting only the winner hides how fragile, or robust, the result is.

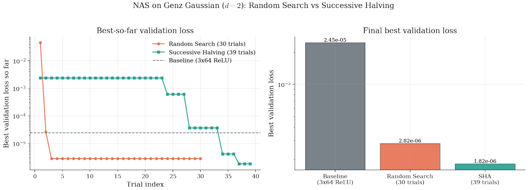
(b) Searching the Architecture Automatically

Fix. Treat the architecture as a variable to **optimize**, and report the protocol:

- random search over the space;
- Bayesian optimization: a GP surrogate of validation loss with an expected-improvement rule;
- Hyperband / successive halving: cheap trials first, then budget the survivors (Bergstra & Bengio 2012; Li et al. 2018).

Remedy

Fix a **compute budget** up front; you cannot search forever. Spend it reproducibly and report the candidate space, not only the winner. Session 2 covers random search and Hyperband.



Search can move validation loss by orders of magnitude before the economic model changes.

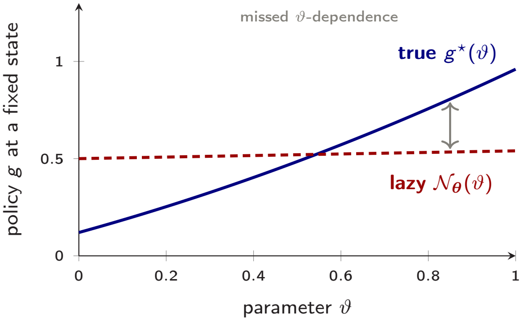
(c) Lazy Learning: Low Residual, Wrong Policy

- **Symptom.** Lazy training is a **typical pitfall of unsupervised** (residual-only) learning.
- It drives the equilibrium residual down while the policy stays inaccurate across the state–parameter domain.

- Fix a state, and vary a *parameter* ϑ (a taste, a tax, a shock size).
- The true policy moves with ϑ; the lazy surrogate stays *flat*, so it gets the level right on average but the **sensitivity** wrong.

Too flat in the parameter

Low loss, weak parameter dependence: the mask is convincing only on average.



At a fixed state, the true policy varies with ϑ (blue); the lazy surrogate is almost flat (red dashed), so it misses how the answer should respond to the parameter.

(c) The Fix: Anchors and Distillation

- **Fix.** Augment the self-supervised residual with a small **semi-supervised distillation** term on a few trusted anchor states:
  $$\ell_{\boldsymbol{\theta}}^{\text{dist}} = \ell_{\boldsymbol{\theta}} + \lambda \sum_{x_a \in \mathcal{A}} \left\| \mathcal{N}_{\boldsymbol{\theta}}(x_a) - p^{\star}(x_a) \right\|^2 .$$

- $\mathcal{A}$: a handful of anchor states.
- $p^{\star}$: reference policies from an accurate solver (perturbation, a low-dimensional solve, or a converged reference run).

Remedy

In the deep-UQ application of Friedl et al. (2023), only **10–20 anchors** stabilize training, with leave-one-out agreement **within** 1%.

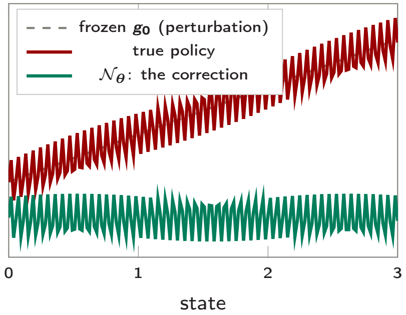
(d) Residual Learning: Freeze and Correct

Idea. Don't ask the network for the *whole* solution. Ask for the **correction** to a cheap baseline:

$$\boldsymbol{g}(\boldsymbol{x}) = \underbrace{\boldsymbol{g}_0(\boldsymbol{x})}_{\text{frozen guess}} + \mathcal{N}_\theta(\boldsymbol{x}).$$

- $\boldsymbol{g}_0$: a cheap guess — steady state, perturbation, or certainty equivalent.
- The net learns only the *small, nonlinear* residual, so **spectral bias bites far less**.
- Sane from iteration zero, and natural for PDE / HJB problems.

→ **The network is a proofreader, not an author.**



The net only fits the small wiggle: frozen guess plus learned correction.

(e) The Small Print of Pathwise Residual Training

Every DEQN solver is **unsupervised**: it drives an equation residual to zero on the states its *own* policy generates,

$$\mathcal{L}_{\mathrm{DEQN}}(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{x}\sim\mu_{\boldsymbol{g}_{\boldsymbol{\theta}}}} \left\| \mathcal{R}(\boldsymbol{x}, \boldsymbol{g}_{\boldsymbol{\theta}}(\boldsymbol{x}), Q^{\boldsymbol{g}_{\boldsymbol{\theta}}}(\boldsymbol{x})) \right\|^2,$$

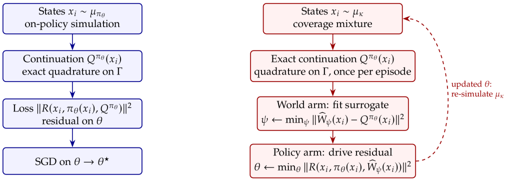
so $\mu_{\boldsymbol{g}_{\boldsymbol{\theta}}}$ is generated **by the candidate policy itself**; $Q^{\boldsymbol{g}_{\boldsymbol{\theta}}}$ is the continuation value under that policy.

- The residual is only ever tested where the current policy already goes.
- A low loss says **nothing** about the states it avoids.
- A wrong policy visits the wrong states, so the error **hides exactly where it is never measured**.
- The loss **confirms itself** — a property of unsupervised training, shared by every method so far, not a DEQN-specific flaw.

(e) EWM: A Disciplined Dreamer

Equilibrium World Models keep the economics and change **where** the residual is imposed.

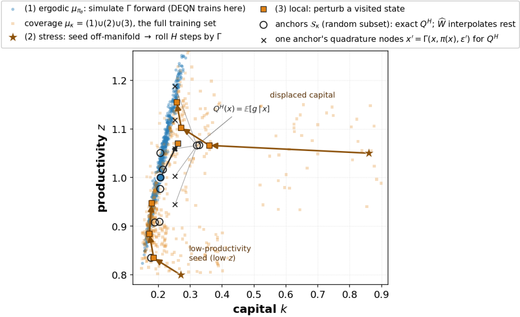
Unsupervised residual solver **EWM**



- **New:** a coverage measure $\mu_{\text{cov}} = c_1 \mu_{g_\theta} + c_2 \mu^{\text{stress}} + c_3 \mu^{\text{local}}$ and a learned continuation $\widehat{Q}_\theta \approx Q^{g_\theta}$.
- **Unchanged:** the law of motion Γ, the residual $\mathcal{R}$, and the exact certificate $\mathcal{R}\big(x, g_\theta(x), Q^{g_\theta}(x)\big)$ on μ_{cov}.
- **EWM is a disciplined dreamer** that certifies with the exact residual.

Equilibrium World Models: Schaab et al. (2026); world-model / Dreamer lineage: Ha & Schmidhuber (2018), Hafner et al. (2023).

(e) What the Audit Buys: Off-Path Credibility



Coverage in (k, z): the ergodic path plus stressed and counterfactual states, each produced by the exact law of motion Γ.

The economics is read in **rare, stressed, counterfactual** regions, exactly where pathwise training is weakest.

- In some **rare-disaster** models the standard deep-learning solver **fails to find a verified solution**, across essentially every random seed.
- With coverage, **EWM solves almost every time**, and certifies the error on the disaster region.

Not a new equilibrium concept, a stronger certificate for the same one.

Part V

Parallelization & HPC

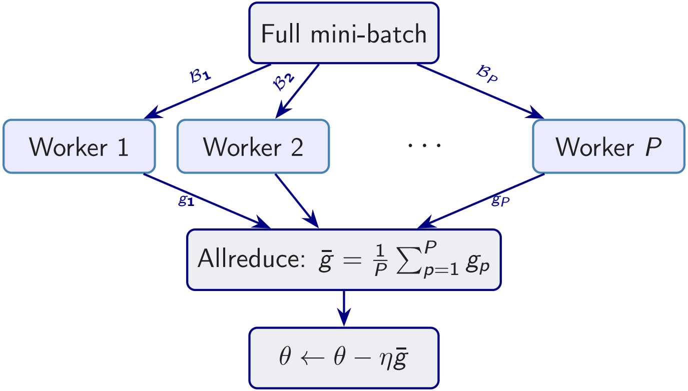
“To pull a bigger wagon, it is easier to add more oxen than to grow a gigantic ox.”

— Gropp, Lusk & Skjellum (1999)

Using MPI: Portable Parallel Programming

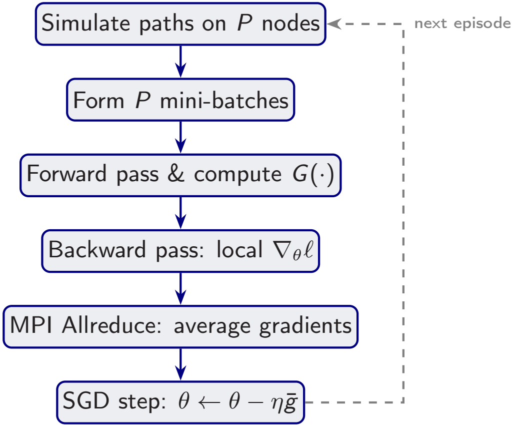
Data Parallelism for DEQNs

Idea: distribute training data across multiple workers (GPUs/CPUs).



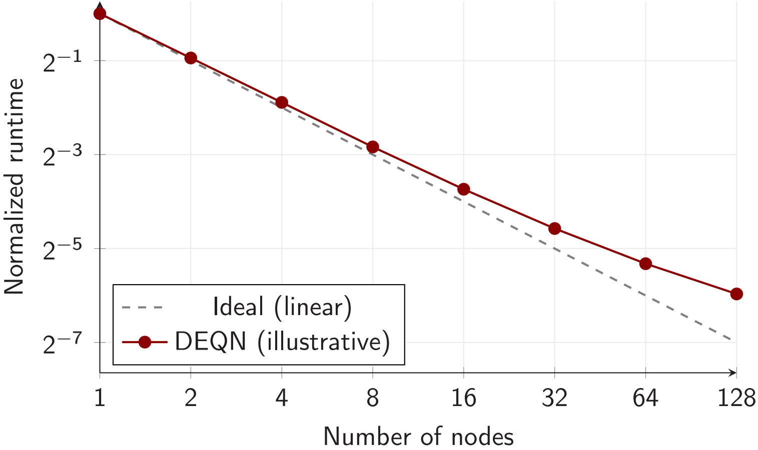
Implemented via **Horovod**/MPI. Each worker computes a local gradient; allreduce averages them.

DEQN Parallelization Scheme



Communication cost $\sim \mathcal{O}(|\theta|)$ per step, independent of data size.

Illustrative Scaling Pattern (Schematic)



Schematic message only: synchronous data parallelism is often close to linear for moderate worker counts; large clusters become increasingly communication-limited.

Preview: the Hands-on Exercises

Exercise notebooks 01_03/01_04 extend Brock–Mirman in two directions:

1. *Endogenous labor* with a non-negativity constraint on leisure, introducing a *Karush–Kuhn–Tucker* multiplier $\mu_t \geq 0$.
2. A *6-period OLG* economy with a borrowing constraint on the young, again KKT with $\mu_t \geq 0$ (a warm-up economy — distinct from the Krueger–Kübler analytic model of Session 5).

Why this matters for DEQNs. A KKT system imposes complementary slackness $\mu \geq 0,\ g \geq 0,\ \mu\, g = 0$ — three conditions that cannot be enforced by a single equation without extra care. *Fischer–Burmeister* packages all three into one residual (Session 5 will use an alternative, the product-form residual $\mu \cdot g = 0$):

$$\phi_{\mathrm{FB}}(a, b) \;=\; a + b - \sqrt{a^2 + b^2}, \qquad \phi_{\mathrm{FB}}(a, b) = 0 \iff a \geq 0,\ b \geq 0,\ a\, b = 0,$$

which is smooth everywhere except at the origin and is fully differentiable by autodiff.

Exercise loss. The notebooks sum a squared *Euler* residual and a squared *FB* residual for each constraint; both are minimized jointly. Details: see Session 4 (IRBC) and the KKT sub-section of the lecture script's IRBC chapter.

Hands-on Notebooks

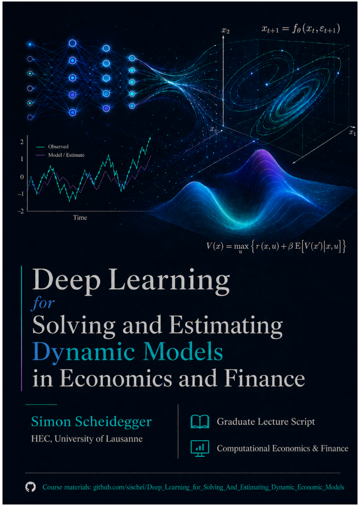
Four notebooks accompany this lecture (plus the optional loss-kernel study `01_05`):

1. `01_01_Brock_Mirman_1972_DEQN.ipynb`
   Deterministic Brock–Mirman, exogenous uniform sampling of states.
2. `01_02_Brock_Mirman_Uncertainty_DEQN.ipynb`
   Stochastic Brock–Mirman ($\varrho > 0$, $\sigma_z > 0$), simulation-based sampling on the ergodic set.
3. `01_03_DEQN_Exercises_Blanks.ipynb` (exercises, blank template for student populate)
4. `01_04_DEQN_Exercises_Solutions.ipynb` (exercises, solved)

Notebooks 01_01–01_02 illustrate the sampling progression: uniform random $\rightarrow$ simulated trajectories. Notebooks 01_03–01_04 add KKT multipliers + Fischer–Burmeister + the 6-period OLG structure previewed on the previous slide.

All notebooks are in `day2/Scheidegger_Kubler/code/01_brock_mirman/`.

Want More Details? The Full Book



Deep Learning for Solving and Estimating Dynamic Models in Economics and Finance

A **book** / graduate lecture script that develops these methods in full.

- **18 lectures**, from foundations to research-level applications.
- Everything is in code — each lecture ships with runnable notebooks.
- In case you want to get more details.

Freely available on GitHub:

`https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models/tree/main`

Questions?

Simon Scheidegger

HEC, University of Lausanne

https://sischei.github.io

Materials: https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

Next up:

Session 4: IRBC — nonlinear DSGE models in 4 to 100+ dimensions

(hands-on exercises: this afternoon's tutorial)